

Kafka Consumer Group Rebalancing SOP

Objective

Stabilize Kafka consumer groups when frequent rebalancing interrupts message processing or increases lag.

Scope

Applicable to Kafka-based applications, streaming platforms, backend services, data engineering pipelines, DevOps teams, and platform teams supporting production or synthetic incident workflows.

Pre-requisites

1. Access to Kafka cluster, namespace or broker host.
2. Access to monitoring dashboards such as Grafana, Prometheus, or cloud monitoring.
3. Kafka CLI tools: kafka-topics, kafka-consumer-groups, kafka-console-producer, and kafka-console-consumer.
4. Permission to view application logs, broker logs, and deployment status.
5. Escalation contact for DevOps, platform, and application owner teams.

Incident Metadata

Field	Value
Ticket ID	T023
Issue	Kafka consumer group rebalancing
Category	Streaming
Service	Kafka
Severity	Low
Status	Resolved
Description	Frequent rebalancing observed
Expected Resolution	Check consumer configuration and heartbeat interval

Related Log Evidence

Log ID	Timestamp	Level	Message	Error Code
L008	5/1/2026 10:04	WARNING	Frequent rebalancing detected	KAFKA_REBALANCE
L043	5/1/2026 10:21	WARNING	Frequent rebalancing detected	KAFKA_REBALANCE

Procedure

1. Confirm rebalance alerts and identify impacted consumer group.
2. Check consumer logs for join group, leave group, poll timeout, or heartbeat failures.
3. Review number of consumer instances against topic partition count.
4. Validate session.timeout.ms, heartbeat.interval.ms, max.poll.interval.ms, and max.poll.records.
5. Check whether consumers are slow due to long processing or blocking calls.
6. Inspect pod restarts, deployment rollouts, and autoscaling events.
7. Tune heartbeat and poll settings and reduce long processing inside poll loop.
8. Monitor rebalance frequency and lag trend after changes.

Common Commands

```
kafka-topics.sh --bootstrap-server <broker>:9092 --describe
kafka-consumer-groups.sh --bootstrap-server <broker>:9092 --describe --group <group>
kafka-broker-api-versions.sh --bootstrap-server <broker>:9092
kubectl get pods -n <namespace> | grep kafka
kubectl logs <kafka-pod> -n <namespace> --tail=100
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
Frequent rebalancing	Heartbeat timeout	Tune heartbeat and session timeout
Rebalance during processing	Long message processing	Increase max.poll.interval.ms or optimize code
Too many consumers	Consumers exceed partitions	Right-size consumer instances
Rebalance during deploy	Rolling restart behavior	Use controlled deployment strategy

Best Practices

1. Keep processing time lower than max poll interval.
2. Avoid aggressive autoscaling for consumer pods.
3. Use cooperative rebalancing when supported.
4. Alert on rebalance frequency, not only consumer lag.

References

Apache Kafka Docs | Internal Logs CSV | Tickets CSV | Monitoring Dashboards | Kafka CLI Runbook